

Department of Computer Engineering and Informatics

Faculty of Science and Technology, Middlesex University Dubai



CST1510

**Programming for Data Communications and
Networks**

Portfolio -1

Student Name: Mukhtar Ali Hussain

MISIS: M00851824

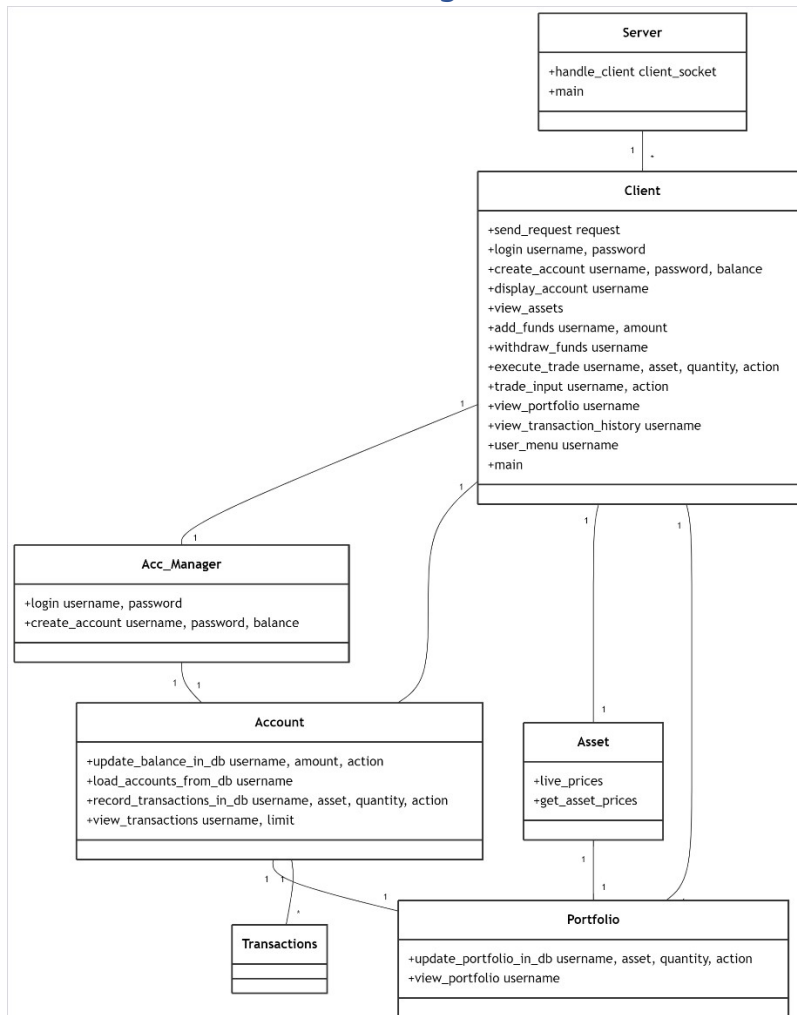
Module Lecturer: Professor Santosh Menon

Table of Contents

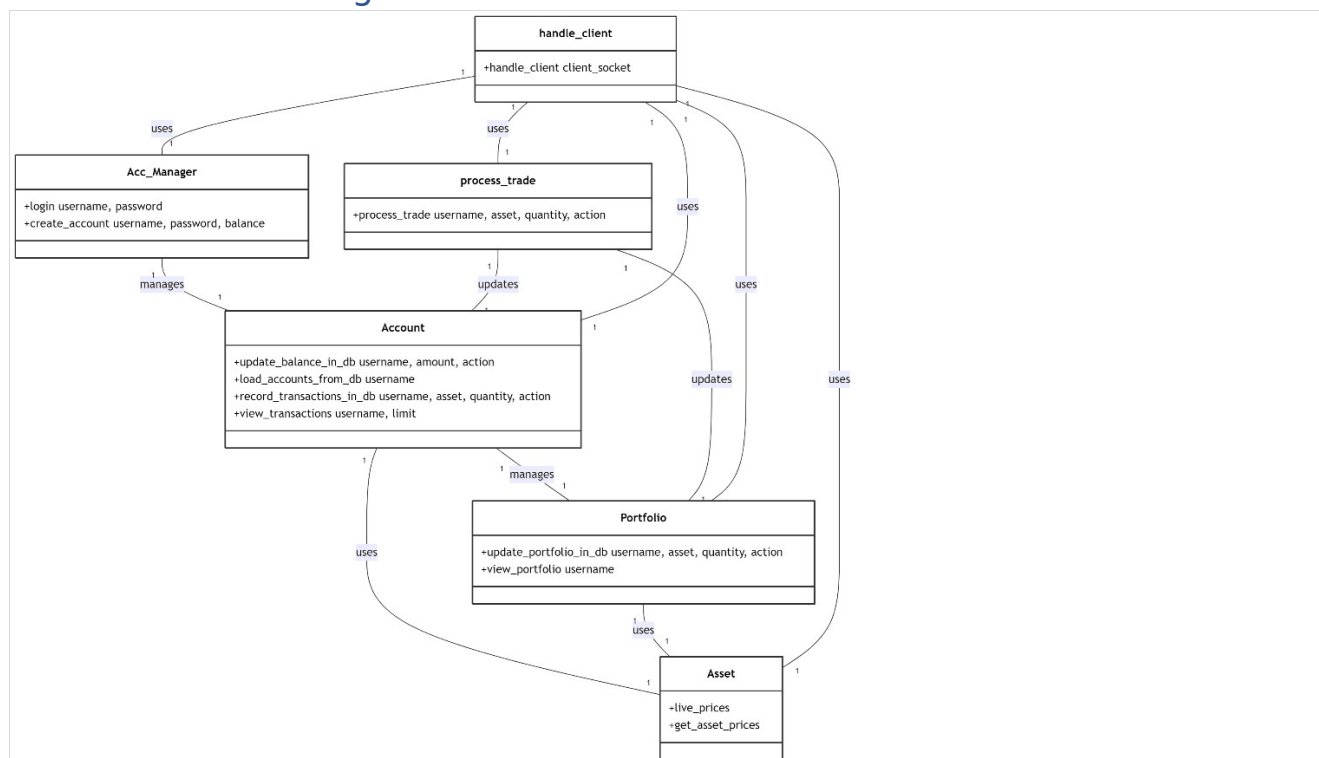
UML Diagram.....	3
Server-CLI UML Diagram:.....	3
Server-GUI UML Diagram:.....	4
Implementation Plan.....	4
Running Instruction.....	5
Python:.....	5
Graphical User Interface (GUI):.....	5
Running the Program:.....	6
Reflection.....	6

UML Diagram

Server-CLI UML Diagram:



Server-GUI UML Diagram:



Implementation Plan

For my approach to building the platform, I simply followed by the PDF and went week by week. Week 6-7 and 7 were pretty simple and straight forward, I created a couple functions to take user input-output and stored it in a text file as requested. However, the account management functions took some time to make it function properly as intended was quite a challenge. Refactoring to Object Orientated Programming was hard to understand in the beginning but became much simple to implement, though after having to completely re-write a lot of the pre-existing functions. After I reached Week 8, client socket communication, I had transferred my file management method to database integration before moving to socket programming to make it easier to work with. The server holds the main logic of the entire program for important functions such as: Login, create account, Save/Update information in the databases, retrieve information from the databases, Process trading where the user either buy's or sell's asset's, storing static Asset prices and using an API to get live prices and client communication. The main function of the client is to display the menus and handle user-input, send requests to the server to login and create a new account, Request account balance, portfolio and transaction history. The Server handles account management, trades, and portfolio updates through several interlinked classes and functions. The Account Manager and Accounts class exists for user authentication, Account creation, Balance update and Transaction logging. The Portfolio class exists modify and retrieve the portfolio. And the Process trade function Verifies the assets, checks for user balance, executing trade and recording transactions

Running Instruction

Python:

Requires Python 3.16 to run the program. To install Python:

- Open Command Line Directory (CMD) and input the following command:
- Get Python from:
- Install PyCharm:

These are some of the methods you can get Python to run the program.

Can be run in Any IDLE. If using PyCharm, Enable the following plugins for importing required libraries: Json, Tkinter (If unavailable or not installed), Socket (if unavailable or not installed), Matplotlib (if unavailable or not installed). By Default, some of these should be pre-installed with the Python package.

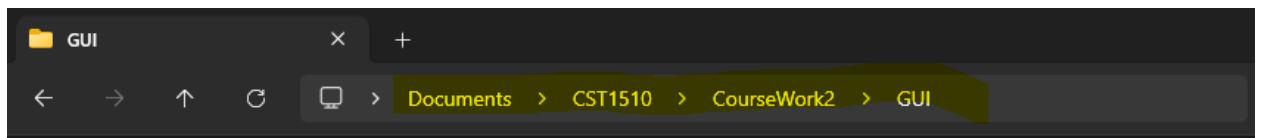
Graphical User Interface (GUI):

The GUI requires background images to run smoothly, however, Tkinter requires full file paths to implement images for the backdrop, replace the lines by pressing "Shift + F3" (#Add alternative options) and Type in the window "C:" and replace the file-paths to where you store the GUI files.

Example:

```
background_image = tk.PhotoImage(file="C:\\Users\\hp\\Documents\\CST1510\\CourseWork2\\GUI\\GUI_backdrop.png")
```

Click on the file-path you currently have the files saved:



Only replace the file paths, not the names of the files themselves to avoid errors.

#NOTE: You need to also add additional "\\"s when replacing the file path to avoid errors.

Running the Program:

PyCharm/Visual Studio Code/IDLE: Simple Locate the "Run Python File" Command (Usually on the top right.).

Command Line:

Change the Directory to where you store the main scripts by entering 'cd "C: File-path\ File_name.exe" '(**Example File Path**)

In order to run the program smoothly, you need to run the server first, Then Client/GUI. Create an Account first, then login and test the available options. **As a measure for any errors encountered:** If GUI Fails to run into any errors not accounted for, Run the Barebones GUI, If Bare-bone GUI does not run properly, Run the CLI (Command Line Interface) Version through Either IDLE's **OR** CMD

Reflection

My experience in developing this project was extremely insightful, revealing my learning habits and areas that need improvement while accelerating my learning curve. The decision to build a cryptocurrency trading platform challenged my problem-solving abilities and logic, testing how I could convert my thought processes into code. I approached the project in phases, taking time to understand weekly requirements and implement them accordingly. The account phase presented challenges, particularly in setting up file handling and inter-function communication. However, with basic Python knowledge and file handling techniques deepened my understanding of conditional statements, which were crucial for functions that depended on user authentication. The trading function proved particularly challenging, requiring about a week to fully comprehend and implement due to its complex requirements and interactions with text files and asset data. Nevertheless, I successfully developed a smooth trade function enabling both buying and selling of assets. Object-Oriented Programming (OOP) concepts were relatively straightforward to understand and implement. I refactored my program to align with question requirements to the best of my abilities. Before transitioning to the client-server design, I integrated database methods first, as I felt that effective data communication was crucial for the program's functionality. This decision was prompted by the inefficiencies and errors encountered with text-based file handling.

These experiences significantly enhanced my understanding of Python, enabling me to approach high-level problems. However, I acknowledge that weaknesses in my foundational knowledge still limit my progress. Despite these challenges, I successfully developed this program with guidance from my tutor, peers, online resources, and self-study materials.

Key challenges included implementing real-time data, particularly in programming API calls and a limit on how many times a call could be made. Socket communication, while on paper sounded simple, proved somewhat challenging to implement. The trading logic was undoubtedly the most difficult aspect, as it depended on multiple functions and I was running into numerous small errors such as incorrect parameters and flawed conditional statement logic. To address issues with calling the database and function misplacements in the socket program, I created a rough UML diagram. This helped me visualise and better understand the program structure, allowing me to refactor by reorganizing functions within existing classes and clarifying functions and what class communicated with what function. This approach helped trace issues back to the database and its implementation in functions like "load_portfolio_from_db" which were resolved through strenuous trial and error. To overcome API implementation challenges, I referred to online documentation and discussed solutions with my more experienced peers.

Throughout this process, I gained valuable insights into code organization and data handling. I refactored my code multiple times to optimize program structure and clarify the flow of functions and classes. Given more time, I would have explored different GUI libraries beyond Tkinter. With additional knowledge, I would have attempted to develop a more comprehensive platform, incorporating features such as live price charts, secure user authentication, a robust backend, asset/portfolio performance tracking, and an enhanced user interface to more closely replicate a professional cryptocurrency trading platform.